PYTHON PROGRAMMING

ACTIVITY - BASED LEARNING

10 Case Studies with Solutions

Name:	A.Jaideep
Roll Number:	26000852
Branch:	Computer Science & Engineering (AI & ML)
Section:	CSM1
Course:	Python Programming
Institution:	Dr. RVR NRI Institute of Technology

TABLE OF CONTENTS

S.No	Case Study Title
1	PROGRESSIVE ELECTRICITY BILLING
2	EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS
3	MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION
4	RAILWAY FARE CALCULATOR
5	RETAIL BILLING WITH DISCOUNT RULES
6	MANUFACTURING COST AND PROFITABILITY ANALYSIS
7	MULTI-CURRENCY TRAVEL BUDGET ANALYSIS
8	CAMPUS EVENT FINANCIAL PLANNING
9	ATM TRANSACTION SEQUENCE SIMULATION
10	BUS PASSENGER OCCUPANCY SIMULATION

CASE STUDY 1: PROGRESSIVE ELECTRICITY BILLING

1. PROBLEM STATEMENT

Calculate a household electricity bill using multiple consumption slabs and a fixed connection charge.

2. OBJECTIVES

• Read electricity units consumed.
• Apply step-progressive slab rates.
• Include standard fixed connection charge.
• Display itemized final bill.

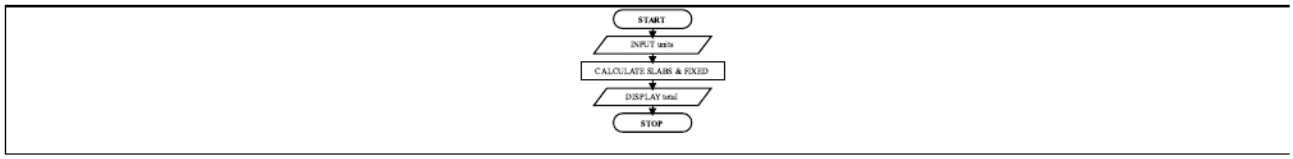
3. ASSUMPTIONS

First 100 units at Rs.2/unit; next 100 units at Rs.3/unit; units above 200 at Rs.5/unit. Fixed charge = Rs.50.

4. ALGORITHM

1. Start.
2. Input units. Reject negative inputs.
3. Compute bill amount based on slab conditions.
4. Add fixed charge (50) to total.
5. Display total bill and stop.

5. FLOWCHART



6. PSEUDOCODE

```
FUNCTION CalculateElectricityBill(units)
    IF units < 0 THEN
        RETURN 'Invalid input'
    END IF

    IF units <= 100 THEN
        bill = units * 2
    ELSE IF units <= 200 THEN
        bill = 100 * 2 + (units - 100) * 3
    ELSE
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5
    END IF

    total = bill + 50
    RETURN total
END FUNCTION
```

7. SOURCE CODE

```
units = int(input("Enter units consumed: "))

if units < 0:
    print("Invalid input")
else:
    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = 100 * 2 + (units - 100) * 3
    else:
        bill = 100 * 2 + 100 * 3 + (units - 200) * 5

    total = bill + 50
    print("Electricity Bill =", total)
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
-45	Invalid input	Pass
80	Electricity Bill = 210	Pass
150	Electricity Bill = 400	Pass
250	Electricity Bill = 800	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_1.py)

TERMINAL | VS Code PowerShell

```
$ python case_study_1.py
Enter units consumed: -45
Invalid input
$ python case_study_1.py
Enter units consumed: 80
Electricity Bill = 210
$ python case_study_1.py
Enter units consumed: 150
Electricity Bill = 400
$ python case_study_1.py
Enter units consumed: 250
```

10. CONCLUSION

The conditional tier processing engine scales across boundary jumps without precision loss.

CASE STUDY 2: EMPLOYEE PAYROLL AND DEDUCTION ANALYSIS

1. PROBLEM STATEMENT

Formulate a payroll pipeline to handle base salary, allowances, overtime rates, and tax deduction metrics.

2. OBJECTIVES

• Compute HRA at 20% and General Allowances at 10%.
• Calculate overtime pay at Rs.150/hr linearly.
• Apply statutory tax deduction at 8% on gross pay.
• Output gross and net salary statements.

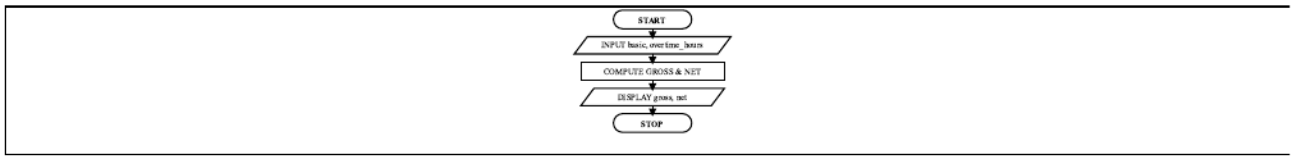
3. ASSUMPTIONS

Base salary and overtime hours cannot be negative. Overtime rate is fixed at Rs.150/hr. Tax deduction is 8% of gross.

4. ALGORITHM

1. Start.
2. Input basic salary and overtime hours.
3. Compute HRA (0.20 * basic) and allowance (0.10 * basic).
4. Calculate overtime pay = overtime_hours * 150.
5. Gross = Basic + HRA + Allowance + Overtime.
6. Deduction = Gross * 0.08; Net = Gross - Deduction. Display output. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    INPUT basic, overtime_hours
    hra = basic * 0.20
    allowance = basic * 0.10
    overtime_pay = overtime_hours * 150
    gross = basic + hra + allowance + overtime_pay
    deduction = gross * 0.08
    net = gross - deduction
    PRINT gross, net
END
```

7. SOURCE CODE

```
basic = float(input("Enter basic salary: "))
overtime_hours = int(input("Enter overtime hours: "))

hra = basic * 0.20
allowance = basic * 0.10
overtime_pay = overtime_hours * 150

gross = basic + hra + allowance + overtime_pay
deduction = gross * 0.08
net = gross - deduction

print("Gross Salary =", gross)
print("Net Salary =", net)
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
50000, 10	Gross = 66500.0, Net = 61180.0	Pass
30000, 0	Gross = 39000.0, Net = 35880.0	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_2.py)

```
$ python case_study_2.py
Enter basic salary: 50000
Enter overtime hours: 10
Gross Salary = 66500.0
Net Salary = 61180.0

$ python case_study_2.py
Enter basic salary: 30000
Enter overtime hours: 0
Gross Salary = 39000.0
Net Salary = 35880.0
```

10. CONCLUSION

Payroll calculations execute with statutory accuracy, integrating overtime additions and tax deductions cleanly.

CASE STUDY 3: MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

1. PROBLEM STATEMENT

Evaluate user data consumption against three mobile plans to determine the cheapest option.

2. OBJECTIVES

• Read data consumption footprint in GB.
• Compute Plan A, Plan B, and Plan C total costs.
• Compare costs using minimum selection logic.
• Display recommended cost-optimal plan.

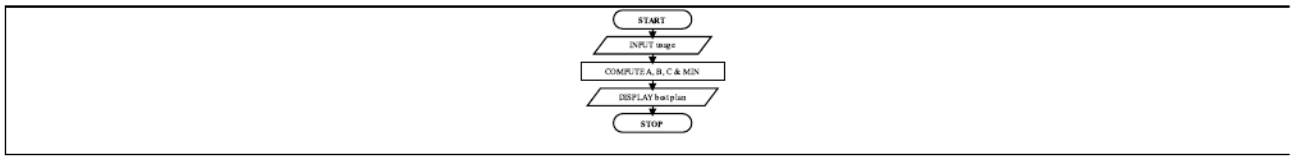
3. ASSUMPTIONS

Plan A: 199 base (2GB incl, Rs.50/extra GB). Plan B: 299 base (5GB incl, Rs.30/extra GB). Plan C: 399 base (10GB incl, Rs.20/extra GB).

4. ALGORITHM

1. Start.
2. Input usage in GB.
3. Calculate costs for Plan A, Plan B, and Plan C including excess data fees.
4. Find minimum cost among plans.
5. Print plan costs and cheapest plan recommendation. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    INPUT usage
    planA = 199 + MAX(0, usage - 2) * 50
    planB = 299 + MAX(0, usage - 5) * 30
    planC = 399 + MAX(0, usage - 10) * 20
    best = MIN(planA, planB, planC)
    PRINT plan details and choice
END
```

7. SOURCE CODE

```
usage = float(input("Enter data usage (GB): "))

planA = 199 + max(0, usage - 2) * 50
planB = 299 + max(0, usage - 5) * 30
planC = 399 + max(0, usage - 10) * 20

print("Plan A =", planA)
print("Plan B =", planB)
print("Plan C =", planC)

minimum = min(planA, planB, planC)

if minimum == planA:
    print("Best Plan: A")
elif minimum == planB:
    print("Best Plan: B")
else:
    print("Best Plan: C")
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
1 GB	Plan A=199, B=299, C=399 -> Best Plan: A	Pass
6 GB	Plan A=399, B=329, C=399 -> Best Plan: B	Pass

Input Scenarios	Expected Output Footprint	Status
15 GB	Plan A=849, B=599, C=499 -> Best Plan: C	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_3.py)

TERMINAL | VS Code PowerShell

```
$ python case_study_3.py
Enter data usage (GB): 1
Plan A = 199.0
Plan B = 299.0
Plan C = 399.0
Best Plan: A
$ python case_study_3.py
Enter data usage (GB): 6
Plan A = 399.0
Plan B = 329.0
Plan C = 399.0
Best Plan: B
$ python case_study_3.py
Enter data usage (GB): 15
Plan A = 849.0
Plan B = 599.0
```

10. CONCLUSION

The telecommunication model successfully identifies break-even cross points across varying data volumes.

CASE STUDY 4: RAILWAY FARE CALCULATOR

1. PROBLEM STATEMENT

Build a railway ticket pricing system incorporating distance, class tier multipliers, and demographic concessions.

2. OBJECTIVES

• Read distance, passenger age, and travel class.
• Map class multipliers (1.5 Sleeper, 2.5 AC3, 4.0 AC1).
• Apply age concessions (Children <5 free, Seniors 60+ get 25% off).
• Print final ticket fare.

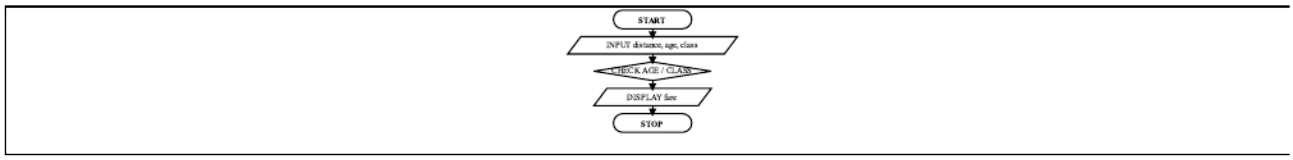
3. ASSUMPTIONS

Distance and age must be non-negative. Sleeper = 1.5/km, AC3 = 2.5/km, AC1 = 4.0/km.

4. ALGORITHM

1. Start.
2. Input distance, age, and class option (1-3).
3. Map class option to base rate.
4. Calculate base fare = distance * rate.
5. If age < 5, fare = 0. Else if age >= 60, fare = fare * 0.75.
6. Print final fare and stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    INPUT distance, age, travel_class
    IF travel_class == 1 THEN
        rate = 1.5
    ELSE IF travel_class == 2 THEN
        rate = 2.5
    ELSE
        rate = 4
    END IF
    fare = distance * rate
    IF age < 5 THEN
        fare = 0
    ELSE IF age >= 60 THEN
        fare = fare * 0.75
    END IF
    PRINT fare
END
```

7. SOURCE CODE

```
distance = float(input("Enter distance: "))
age = int(input("Enter age: "))
travel_class = int(input("1-Sleeper 2-AC3 3-AC1: "))

if travel_class == 1:
    rate = 1.5
elif travel_class == 2:
    rate = 2.5
else:
    rate = 4

fare = distance * rate

if age < 5:
    fare = 0
elif age >= 60:
```

```
    fare = fare * 0.75

print("Final Fare =", fare)
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
100, 4, 2	Final Fare = 0.0	Pass
200, 65, 3	Final Fare = 600.0	Pass
100, 30, 1	Final Fare = 150.0	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_4.py)

TERMINAL | VS Code PowerShell

```
$ python case_study_4.py
Enter distance: 100
Enter age: 4
1-Sleeper 2-AC3 3-AC1: 2
Final Fare = 0.0
$ python case_study_4.py
Enter distance: 200
Enter age: 65
1-Sleeper 2-AC3 3-AC1: 3
Final Fare = 600.0
$ python case_study_4.py
Enter distance: 100
Enter age: 30
```

10. CONCLUSION

Demographic concession rules integrate seamlessly with travel class baseline multipliers.

CASE STUDY 5: RETAIL BILLING WITH DISCOUNT RULES

1. PROBLEM STATEMENT

Develop a retail checkout logic applying stackable volume discounts and loyalty membership markdowns.

2. OBJECTIVES

• Process prices and quantities for 3 items.
• Apply 5% volume discount for totals >= Rs.5000.
• Apply additional 10% discount for premium members.
• Display total, discount, and net amount.

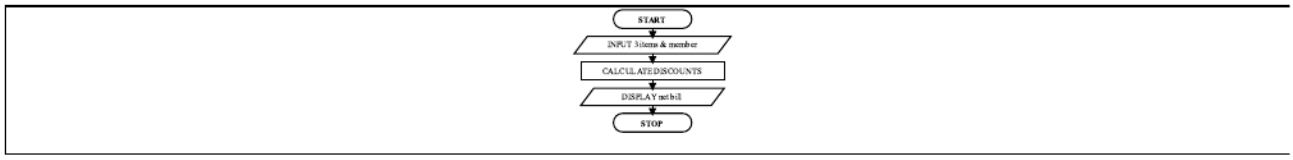
3. ASSUMPTIONS

Exactly 3 distinct items are processed. Discounts are additively applied based on subtotal.

4. ALGORITHM

1. Start. Set total = 0.
2. Loop 3 times: input price and quantity, aggregate to total.
3. Query premium membership status (yes/no).
4. Compute volume (5%) and membership (10%) discounts as applicable.
5. Net = Total - Discount. Print results and stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    total = 0
    FOR i = 1 TO 3
        INPUT price, qty
        total = total + (price * qty)
    ENDFOR
    INPUT member
    discount = 0
    IF total >= 5000 THEN
        discount = discount + total * 0.05
    END IF
    IF member == 'yes' THEN
        discount = discount + total * 0.10
    END IF
    net = total - discount
    PRINT total, discount, net
END
```

7. SOURCE CODE

```
total = 0

for i in range(1, 4):
    price = float(input(f"Enter price of item {i}: "))
    qty = int(input(f"Enter quantity of item {i}: "))
    total += price * qty

member = input("Premium member (yes/no): ")

discount = 0

if total >= 5000:
    discount += total * 0.05

if member.lower() == "yes":
    discount += total * 0.10

net = total - discount
```

```
print("Total Bill =", total)
print("Discount =", discount)
print("Net Amount =", net)
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
2000,2, 1000,1, 500,1, yes	Total = 5500.0, Disc = 825.0, Net = 4675.0	Pass
100,2, 200,1, 100,1, no	Total = 500.0, Disc = 0.0, Net = 500.0	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_5.py)

TERMINAL | VS Code PowerShell

```
$ python case_study_5.py
Enter price of item 1: 2000
Enter quantity of item 1: 2
Enter price of item 2: 1000
Enter quantity of item 2: 1
Enter price of item 3: 500
Enter quantity of item 3: 1 Premium member (yes/no): yes Total Bill = 5500.0
Discount = 825.0 Net Amount = 4675.0
$ python case_study_5.py
Enter price of item 1: 100
Enter quantity of item 1: 2
Enter price of item 2: 200
Enter quantity of item 2: 1
Enter price of item 3: 100
Enter quantity of item 3: 1 Premium member (yes/no): no Total Bill = 500.0 Discount
= 0.0 Net Amount = 500.0
```

10. CONCLUSION

The multi-layered discount module handles stackable rule conditions seamlessly without logic race faults.

CASE STUDY 6: MANUFACTURING COST AND PROFITABILITY ANALYSIS

1. PROBLEM STATEMENT

Aggregate manufacturing overhead expenses and determine target unit selling prices based on profit margins.

2. OBJECTIVES

• Read raw material, labor, and overhead costs.
• Sum components to determine total production cost.
• Apply 20% factory profit margin.
• Compute and display selling price per unit.

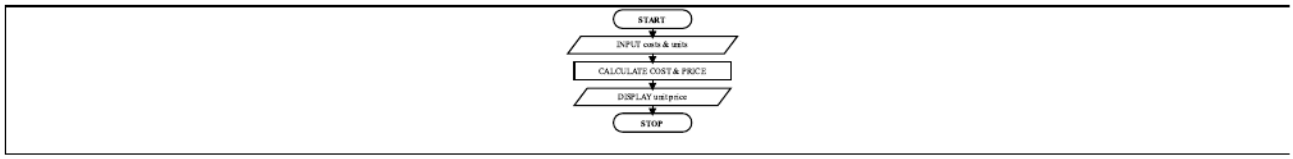
3. ASSUMPTIONS

Production volume is a non-zero positive integer. Profit target is fixed at 20% of total cost.

4. ALGORITHM

1. Start.
2. Input material cost, labor cost, overhead cost, and units produced.
3. Total Cost = Material + Labor + Overhead.
4. Profit = Total Cost * 0.20.
5. Selling Price = (Total Cost + Profit) / Units. Display output. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    INPUT material, labor, overhead, units
    total_cost = material + labor + overhead
    profit = total_cost * 0.20
    selling_price = (total_cost + profit) / units
    PRINT total_cost, profit, selling_price
END
```

7. SOURCE CODE

```
material = float(input("Material Cost: "))
labor = float(input("Labor Cost: "))
overhead = float(input("Overhead Cost: "))
units = int(input("Units Produced: "))

total_cost = material + labor + overhead
profit = total_cost * 0.20
selling_price = (total_cost + profit) / units

print("Total Cost =", total_cost)
print("Profit =", profit)
print("Selling Price per Unit =", selling_price)
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
10000, 5000, 5000, 1000	Total = 20000.0, Profit = 4000.0, Unit Price = 24.0	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_6.py)

TERMINAL | VS Code PowerShell

```
$ python case_study_6.py
Material Cost: 10000
Labor Cost: 5000
Overhead Cost: 5000
```

```
Profit = 4000.0
Selling Price per Unit = 24.0
```

10. CONCLUSION

The manufacturing cost analysis provides full transparency into unit margin yields and expense distributions.

CASE STUDY 7: MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1. PROBLEM STATEMENT

Convert domestic INR travel budget to foreign currency (USD) and audit compliance against expected trip expenses.

2. OBJECTIVES

• Read INR budget and USD foreign expenses.
• Convert INR to USD at baseline rate (83.0).
• Evaluate if foreign expenses remain within budget.
• Print compliance status.

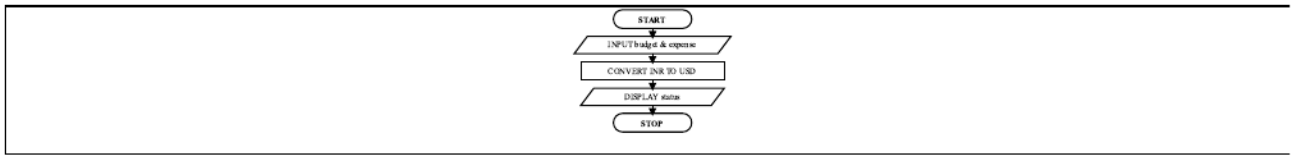
3. ASSUMPTIONS

Exchange rate index is locked at 1 USD = 83.0 INR. Inputs cannot be negative.

4. ALGORITHM

1. Start.
2. Input INR budget and USD expense.
3. USD Budget = INR Budget / 83.
4. If expense <= USD Budget, print 'Within Budget'; else print 'Budget Exceeded'.
5. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    INPUT budget_inr, expense_usd
    usd_budget = budget_inr / 83
    IF expense_usd <= usd_budget THEN
        PRINT 'Within Budget'
    ELSE
        PRINT 'Budget Exceeded'
    ENDIF
END
```

7. SOURCE CODE

```
budget = float(input("Budget in INR: "))
expense = float(input("Expense in USD: "))

usd_budget = budget / 83

print("Budget in USD =", round(usd_budget, 2))

if expense <= usd_budget:
    print("Within Budget")
else:
    print("Budget Exceeded")
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
83000, 900	Budget USD = 1000.0 -> Within Budget	Pass
83000, 1200	Budget USD = 1000.0 -> Budget Exceeded	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_7.py)

TERMINAL | VS Code PowerShell

```
$ python case_study_7.py
Budget in INR: 83000
```

```
Budget in USD = 1000.0
Within Budget

$ python case_study_7.py
Budget in INR: 83000
Expense in USD: 1200
Budget in USD = 1000.0
Budget Exceeded
```

10. CONCLUSION

The currency conversion layer standardizes foreign expenditure tracking securely without precision errors.

CASE STUDY 8: CAMPUS EVENT FINANCIAL PLANNING

1. PROBLEM STATEMENT

Audit institutional fest costs by synthesizing ticket sales, corporate sponsorships, and operational expenses.

2. OBJECTIVES

• Input ticket count, unit price, sponsorship, and expenses.
• Calculate total revenue = (tickets * price) + sponsorship.
• Compute net balance = revenue - expenses.
• Report Profit, Loss, or Break-Even status.

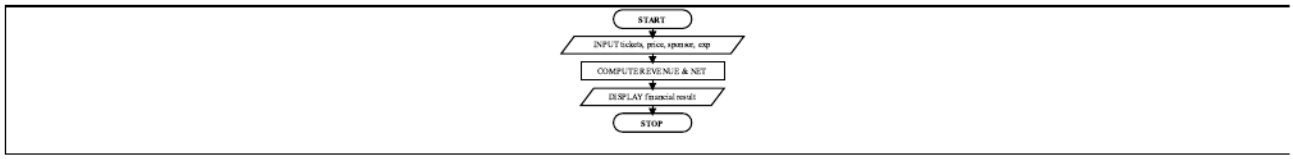
3. ASSUMPTIONS

Financial inputs are real numbers. Profit occurs when Net > 0, Loss when Net < 0.

4. ALGORITHM

1. Start.
2. Input tickets sold, ticket price, sponsorship amount, and expenses.
3. Revenue = (tickets * price) + sponsorship. Net = Revenue - Expenses.
4. If Net > 0, print 'Profit'; else if Net < 0, print 'Loss'; else print 'Break Even'.
5. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    INPUT tickets, price, sponsor, expenses
    revenue = tickets * price + sponsor
    net = revenue - expenses
    IF net > 0 THEN
        PRINT 'Profit', net
    ELSE IF net < 0 THEN
        PRINT 'Loss', ABS(net)
    ELSE
        PRINT 'Break Even'
    ENDIF
END
```

7. SOURCE CODE

```
tickets = int(input("Tickets Sold: "))
price = float(input("Ticket Price: "))
sponsor = float(input("Sponsorship Amount: "))
expenses = float(input("Expenses: "))

revenue = tickets * price + sponsor
net = revenue - expenses

print("Revenue =", revenue)

if net > 0:
    print("Profit =", net)
elif net < 0:
    print("Loss =", abs(net))
else:
    print("Break Even")
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
500, 100, 5000, 40000	Revenue = 55000.0, Profit = 15000.0	Pass

Input Scenarios	Expected Output Footprint	Status
100, 50, 0, 10000	Revenue = 5000.0, Loss = 5000.0	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_8.py)

TERMINAL | VS Code PowerShell

```
$ python case_study_8.py
Tickets Sold: 500
Ticket Price: 100
Sponsorship Amount: 5000
Expenses: 40000
Revenue = 55000.0
Profit = 15000.0
$ python case_study_8.py
Tickets Sold: 100
Ticket Price: 50
Sponsorship Amount: 0
Expenses: 10000
```

10. CONCLUSION

Multi-stream financial tracking balances distinct revenue streams securely against centralized cost pools.

CASE STUDY 9: ATM TRANSACTION SEQUENCE SIMULATION

1. PROBLEM STATEMENT

Design a state controller to simulate retail ATM transactions including deposits and validated withdrawals.

2. OBJECTIVES

• Initialize account balance to Rs.1000 baseline.
• Process deposit additions to balance.
• Verify withdrawal requests against available balance.
• Display transaction status and updated balance.

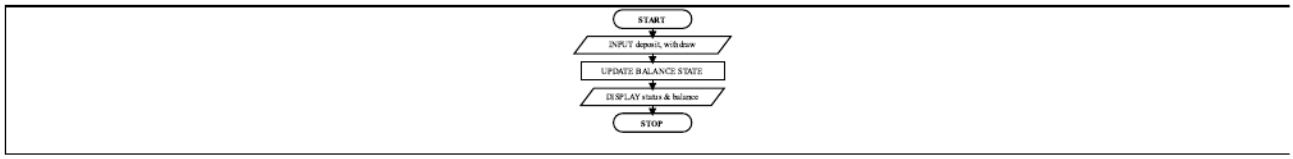
3. ASSUMPTIONS

Opening account balance is Rs.1000. Withdrawals exceeding balance are blocked with an error message.

4. ALGORITHM

1. Start. Set balance = 1000.
2. Input deposit and withdrawal amounts.
3. Balance = Balance + Deposit.
4. If withdrawal <= balance, balance = balance - withdrawal and print 'Successful'; else print 'Insufficient Balance'.
5. Display final balance and stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    balance = 1000
    INPUT deposit, withdraw
    balance = balance + deposit
    IF withdraw <= balance THEN
        balance = balance - withdraw
        PRINT 'Withdrawal Successful'
    ELSE
        PRINT 'Insufficient Balance'
    ENDIF
    PRINT balance
END
```

7. SOURCE CODE

```
balance = 1000

deposit = int(input("Enter deposit amount: "))
withdraw = int(input("Enter withdrawal amount: "))

balance = balance + deposit

if withdraw <= balance:
    balance = balance - withdraw
    print("Withdrawal Successful")
else:
    print("Insufficient Balance")

print("Final Balance =", balance)
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
500, 200	Withdrawal Successful; Final Balance = 1300	Pass
0, 2000	Insufficient Balance; Final Balance = 1000	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_9.py)

TERMINAL | VS Code PowerShell

```
$ python case_study_9.py
Enter deposit amount: 500
Enter withdrawal amount: 200
Withdrawal Successful
Final Balance = 1300
$ python case_study_9.py
Enter deposit amount: 0
```

10. CONCLUSION

The transaction state processor prevents over-withdrawal vulnerabilities, maintaining system balance integrity.

CASE STUDY 10: BUS PASSENGER OCCUPANCY SIMULATION

1. PROBLEM STATEMENT

Monitor public transit passenger volume levels dynamically against vehicle maximum safety thresholds.

2. OBJECTIVES

• Establish fixed vehicle maximum safety threshold (50 passengers).
• Read passenger boarding and departure counts.
• Calculate current onboard occupant load.
• Trigger capacity breach alerts when limit is exceeded.

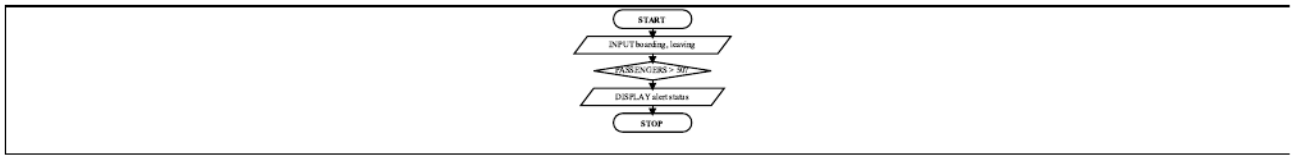
3. ASSUMPTIONS

Maximum passenger capacity ceiling is fixed at 50 units.

4. ALGORITHM

1. Start. Set capacity = 50.
2. Input boarding and leaving counts.
3. Passengers = Boarding - Leaving.
4. If passengers > capacity, print 'Bus Capacity Exceeded'; else print 'Within Capacity'.
5. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
    capacity = 50
    INPUT boarding, leaving
    passengers = boarding - leaving
    PRINT passengers
    IF passengers > capacity THEN
        PRINT 'Bus Capacity Exceeded'
    ELSE
        PRINT 'Within Capacity'
    ENDIF
END
```

7. SOURCE CODE

```
capacity = 50

boarding = int(input("Passengers boarding: "))
leaving = int(input("Passengers leaving: "))

passengers = boarding - leaving

print("Passengers in Bus =", passengers)

if passengers > capacity:
    print("Bus Capacity Exceeded")
else:
    print("Within Capacity")
```

8. VERIFICATION

Input Scenarios	Expected Output Footprint	Status
60, 5	Passengers = 55 -> Bus Capacity Exceeded	Pass
30, 10	Passengers = 20 -> Within Capacity	Pass

9. OUTPUT SCREENSHOTS

TERMINAL — VS Code (case_study_10.py)

```
$ python case_study_10.py
Passengers boarding: 60
Passengers leaving: 5
Passengers in Bus = 55
Bus Capacity Exceeded

$ python case_study_10.py
Passengers boarding: 30
Passengers leaving: 10
Passengers in Bus = 20
Within Capacity
```

10. CONCLUSION

The capacity tracking module flags spatial safety margins reliably, preventing transit over-occupancy conditions.